

# Complete Firebase Project Change Guide

Flutter Application • Windows / PowerShell • Firebase CLI • FlutterFire CLI

## Purpose

This consolidated guide combines the two provided Firebase/FlutterFire setup documents into one client-ready procedure for changing a Flutter application's Firebase project.

## Before You Start

**Required:** Access to the new Firebase project with the necessary permissions. The procedure changes the application's Firebase configuration; it does **not** migrate existing Firebase data.

## 1. Identify the Flutter Application Folder

The repository may contain the Flutter application inside a subfolder such as `frontend`. The FlutterFire configuration command must be run from the actual Flutter application directory—the directory containing `pubspec.yaml`.

If the command is run from the repository root, FlutterFire can return:

`FlutterAppRequiredException: The current directory does not appear to be a Flutter application project.`

Move into the Flutter application folder first:

```
cd frontend
```

Replace `frontend` if your Flutter app is stored in another directory.

## 2. Delete the Existing Firebase Configuration Files

Before connecting the application to the new Firebase project, delete these existing configuration files:

| File                                   | Purpose                                              |
|----------------------------------------|------------------------------------------------------|
| <code>.firebaserc</code>               | Existing Firebase project/reference configuration.   |
| <code>firebase.json</code>             | Existing Firebase CLI configuration.                 |
| <code>lib/firebase_options.dart</code> | Generated Firebase options for the existing project. |

**Do not delete the entire `lib` folder or other application source files.**

PowerShell command to remove the three files:

```
Remove-Item -Path .firebaserc, firebase.json, lib/firebase_options.dart -ErrorAction SilentlyContinue
```

## 3. Check Firebase CLI

Open the project in VS Code and open a PowerShell terminal. Check that Firebase CLI is installed:

```
firebase --version
```

## 4. Log In to Firebase

Authenticate with the Google account that has access to the **new Firebase project**:

```
firebase login
```

A browser window opens for Google authentication. The setup reference also notes that Firebase may ask about Gemini in Firebase and CLI usage-data collection; those prompts can be declined.

## 5. Activate FlutterFire CLI

Install/activate FlutterFire CLI globally using Dart:

```
dart pub global activate flutterfire_cli
```

On Windows, the Dart Pub executable directory may not be on PATH. The provided setup reference identifies the default location as:

```
C:\Users\<YourUsername>\AppData\Local\Pub\Cache\bin
```

Add this directory to the system PATH if the FlutterFire executable cannot be resolved.

## 6. If FlutterFire CLI Is Not Working Correctly

The original setup log showed an inactive package and an executable-resolution error. If this occurs, perform a clean reactivation:

```
dart pub global deactivate flutterfire_cli
dart pub global activate --overwrite flutterfire_cli
```

The setup log showed that this form may fail:

```
dart pub global run flutterfire_cli configure
```

with an error such as:

Could not find bin\flutterfire\_cli.dart in package flutterfire\_cli.

Use the explicit executable form below instead.

## 7. Configure the Application with the New Firebase Project

**Important:** Run the following command from the Flutter application folder containing `pubspec.yaml`.

```
dart pub global run flutterfire_cli:flutterfire configure
```

The CLI will display Firebase projects available to the logged-in account. Select the **new Firebase project** that should be connected to the application.

## 8. Select the Required Platforms

When prompted for supported platforms, select the platforms required by the application. The provided setup used:

android, ios, web

## 9. Generated Firebase Configuration

FlutterFire registers the required Firebase applications for the selected platforms and generates:

```
lib/firebase_options.dart
```

The setup reference showed Android, iOS, and Web applications being registered and a new `firebase_options.dart` being generated.

## 10. Verify the New Configuration

1. Confirm that `lib/firebase_options.dart` has been generated again.
2. Confirm that the newly selected Firebase project was used.
3. Run the Flutter application.
4. Test Firebase features used by the application, such as Authentication, Firestore/Database, Storage, or other configured services.

## 11. Complete Command Reference

Use this as the consolidated command sequence. Adjust frontend to the actual Flutter app folder if necessary.

```
cd frontend
```

```
firebase --version
firebase login
```

```
Remove-Item -Path .firebaserc, firebase.json, lib/firebase_options.dart -ErrorAction SilentlyContinue
```

```
dart pub global activate flutterfire_cli
```

```
# Only if FlutterFire CLI activation/executable resolution is broken:
```

```
dart pub global deactivate flutterfire_cli
dart pub global activate --overwrite flutterfire_cli

# Configure with the new Firebase project:
dart pub global run flutterfire_cli:flutterfire configure
```

## 12. Troubleshooting

### Error: No active package flutterfire\_cli

Run `dart pub global activate flutterfire_cli`.

### Error: Could not find bin\flutterfire\_cli.dart

Deactivate and reactivate FlutterFire CLI with `--overwrite`, then use `dart pub global run flutterfire_cli:flutterfire configure`.

### Error: FlutterAppRequiredException

Navigate to the Flutter application folder containing `pubspec.yaml`, then run the configure command again.

### FlutterFire executable not recognized

Check that the Dart Pub Cache bin directory is on PATH. The provided reference gives `C:\Users\<YourUsername>\AppData\Local\Pub\Cache\bin` as the Windows location.

## 13. Firebase Data Migration

**Important:** Changing the Firebase project configuration does not automatically move data from the old project to the new one.

If existing Firebase Authentication users, Firestore/Realtime Database data, Storage files, or other resources need to be retained, those resources must be migrated separately. The provided files do not contain a data-migration procedure, so this guide does not specify one.

## 14. Final Client Checklist

- Access to the new Firebase project confirmed
- Flutter application folder identified
- `.firebaserc` deleted
- `firebase.json` deleted
- `lib/firebase_options.dart` deleted
- Firebase CLI version checked
- Firebase login completed
- FlutterFire CLI activated
- FlutterFire CLI reactivated with `--overwrite` if necessary
- Configuration command run from the Flutter app folder
- New Firebase project selected
- Required platforms selected
- New `lib/firebase_options.dart` generated
- Application launched successfully
- Firebase functionality tested
- Any required data migration handled separately

Source basis: consolidated from the two Firebase/FlutterFire setup PDFs provided for this task.